CSE 144 Final Project Report

Transfer Learning Challenge

Group Members

- Archita Srikrishnan (CruzID: 2021831)

Spring 2026

---

1 Introduction

The goal of this project was to sort food images into 100 categories using transfer learning. The dataset contained 10 training examples per class, which made transfer learning more appropriate than training a CNN from scratch.

Transfer learning was better because pretrained models already contain strong low-level and high-level visual features learned from large datasets such as ImageNet. By fine-tuning a pretrained network, I saw significantly better performance while reducing training time and overfitting.

My final approach used EfficientNet-B2 pretrained on ImageNet through TorchVision. I fine-tuned the network using a two-stage training process and used data augmentation techniques to improve generalization. My best Kaggle submission achieved 70.9% test accuracy.

---

2 Dataset

The dataset consisted of 100 food classes with 10 training images per class, adding up to 1000 total training images. The test set contained 1036 unlabeled images.

Each folder name directly corresponded to its class label. Folder 0 mapped to label 0, folder 1 mapped to label 1, and so on through label 99. This mapping was important because incorrect label ordering would reduce accuracy.

I resized images to 260×260 before training. Input images were normalized using ImageNet mean and standard deviation values.

The data augmentation techniques I used were:

- Random resized cropping
- Horizontal flipping
- Random rotation
- Color jitter

These augmentations helped reduce overfitting.

---

3 Implementation

3.1 Model

The project used EfficientNet-B2 from TorchVision with pretrained ImageNet weights.

EfficientNet-B2 was chosen because it provides a strong balance between model size and classification accuracy. Compared to a simple CNN, EfficientNet has better feature extraction capabilities while remaining computationally manageable.

The original classifier layer was replaced with a fully connected layer containing 100 output classes that correspond to the food categories.

The training process used a two-stage fine-tuning strategy:

1. Freeze the EfficientNet backbone and train only the classifier head.
2. Unfreeze the entire network and fine-tune all layers using a smaller learning rate.

This approach allowed the model to slowly adapt to the food classification task while absorbing useful pretrained features.

3.2 Training

The model used cross-entropy loss with label smoothing.

Optimizer:

- AdamW

Training hyperparameters:

- Batch size: 16
- Image size: 260×260
- Head training epochs: 5
- Fine-tuning epochs: 25
- Learning rate (head training): 1e-3
- Learning rate (fine-tuning): 1e-5
- Weight decay: 1e-4

The project used:

- Python
- PyTorch
- TorchVision
- NumPy
- Pandas
- Matplotlib

Training was performed on Apple Silicon using the PyTorch MPS backend.

---

4 Experiments

My first setup used EfficientNet-B2 with standard transfer learning and basic augmentation. This Kaggle accuracy was approximately 65%.

I tested different changes, such as:

- Increasing fine-tuning epochs
- Different learning rates
- Stronger augmentations
- EfficientNet-B3

The best validation checkpoint was saved as best_model.pt.

The most successful improvement came from increasing data augmentation and increasing fine-tuning epochs, which improved Kaggle accuracy to 70.9%.

Some larger architectural and augmentation changes reduced accuracy, because they overfit more on the small dataset.

---

5 Results

The final EfficientNet-B2 model achieved approximately 70.9% accuracy on the Kaggle public leaderboard.

Training and validation accuracy increased steadily during fine-tuning. Data augmentation and transfer learning significantly improved performance compared to training a model from scratch.

The model generalized well, even though it had a limited number of training examples per class.

The final submission used:

- EfficientNet-B2
- Transfer learning
- Data augmentation
- Fine-tuned pretrained weights

---

6 Discussion

Transfer learning was an efficient strategy for this project because the dataset was too small to train a deep network from scratch. EfficientNet-B2 provided strong pretrained weights that transferred well to the food classification task.

Data augmentation also helped reduce overfitting. Some of my trials with deeper models and stronger augmentation reduced performance, because the dataset size was too small to support more intense fine-tuning.

Future improvements could include:

- Model ensembling
- K-fold cross-validation
- ConvNeXt or Vision Transformer backbones

---

7 Reproducibility

A fixed random seed of 42 was used for reproducibility.

Required packages:

- torch
- torchvision
- numpy
- pandas
- pillow
- matplotlib

Training command:

python cse144final.py

The script automatically:

1. Trains the model
2. Saves the best weights as best_model.pt
3. Generates submission.csv

---

8 Team Contributions

Since I worked on my own, I worked on all aspects of the project.

---

9 References

1. TorchVision Model Documentation
   https://pytorch.org/vision/stable/models.html
2. PyTorch Documentation
   https://pytorch.org/docs/stable/index.html